

Type System & Semantics

Cast

- `static_cast` – Compiler-checked → CRTP, enum conversion
- `dynamic_cast` – Runtime-checked, vtable lookup → Not used (avoid in embedded)
- `reinterpret_cast` – Unchecked, zero cost → `Reg<Addr> int` → pointer
- `const_cast` – Unchecked, zero cost → can cast away `const` but not used
- `int` will cast to `uint` when added together

lvalue vs rvalue

- `lvalue` features persistent address in mem so can be pointed to
- `rvalue` temporary lives in `CPU register` during evaluation and dies at the end of expression ;
- & test : & get the address outof `lvalue` while `rvalue` cannot

```
int x = 5;           // x is an lvalue – named, has address, persists
                    // 5 is an rvalue – temporary, no lasting identity

process(x);          // binds to int& (lvalue ref)
process(5);          // binds to int&& (rvalue ref)
process(std::move(x)); // std::move CASTS x to rvalue ref – does NOTHING at runtime
```

Syntax	Name	Binds to
<code>T&</code>	lvalue reference	alias to variable
<code>const T&</code>	const lvalue reference	read-only alias (accepts rvalue too)
<code>T&&</code>	rvalue reference	binds to temporaries

`std::move` – cast, not copy

- `std::move(x)` is equivalent to `static_cast<T&&>(x)` produces `T&&` – a cast, not an operation
- The move constructor then *steals* heap pointers from the rvalue
- After `std::move(x)`, `x` is still a valid object in a "moved-from" state (can be destroyed or reassigned, but content is unspecified)

Trivial vs non-trivial types

- **Trivial** – no constructor, no destructor, no virtual methods. Raw bytes. `memcpy` is safe.
 - `int`, `double`, `float`, `char`, `int[N]`, `struct { double x, y; }`
 - `int x = 5;` → 4 bytes on stack.
 - `int* p = new int(5);` → 8-byte pointer on stack + 4-byte int on heap.
 - The `new` version requires manual `delete` or a `smart pointer` to avoid leaking.
- **Non-trivial (resource-managing)** – `ctor` allocates heap, `dtor` frees it.
 - `std::string`, `std::vector<T>`, `std::unique_ptr<T>`

- `memcpy` blindly copies stack bytes (ptr, size, cap) → both `dtor` of src and dest call `free()` → double free
- **Virtual types** – `memcpy` → wrong `vptr` → virtual calls jump to garbage.
 - **ctor sequence** for vtable
 - `vptr` lives inside each object instance (stack or heap, usually first hidden field).
 - `vtable` lives in read-only program data (`.rodata`) and is shared per dynamic type.
 - 1. **base ctor** runs first and sets `vptr` → `Base::vtable`.
 - 2. **derived ctor** runs and overwrites `vptr` → `Derived::vtable`. This ensures virtual fns during base ctor dispatch to base methods (not yet-constructed derived), and after full construction, `vptr` points to the final derived vtable.
- A class with `virtual` has a hidden `vptr` pointing to its vtable. `memcpy` copies raw bytes (including `vptr`) without invoking the `ctor` that correctly sets `vptr` for the **derived** actual type.

```
#include <type_traits>
static_assert(std::is_trivially_copyable_v<double>);           // ✓
static_assert(std::is_trivially_copyable_v<int[4]>);           // ✓
static_assert(!std::is_trivially_copyable_v<std::string>);     // ✓ not trivial
```

Keyword reference

Keyword	Affects	Purpose
<code>const</code> (on variable)	compiler	prevents modification, enables placing in <code>.rodata</code>
<code>const</code> (after method)	compiler	method can be called on <code>const&</code> objects; use <code>mutable</code> for exceptions
<code>volatile</code>	compiler	prevents read/write elimination – every access emits a real load/store
<code>inline</code>	linker symbol	changes symbol from strong (T) to weak (W) – allows multiple definitions
<code>constexpr</code>	compiler	enables compile-time evaluation; implies <code>inline</code>
<code>static</code> (in function)	compiler/linker	variable persists across calls; stored in <code>.bss/.data</code> , not stack
<code>static</code>	linker	internal linkage – symbol visible only in this TU (t in nm)
<code>static</code> (class member)	linker	one instance shared across all objects <-> not tied to <code>this</code>
<code>mutable</code>	compiler	allows modification of this field even in <code>const</code> methods (e.g., caches, mutexes)
<code>atomic</code>	compiler/hw	thread-safe atomic operations + memory ordering (lock-free only for some types/architectures)

Keyword	Affects	Purpose
global variable	linker	external linkage by default – visible across all TUs; stored in <code>.data/.bss</code> ; use <code>extern</code> to declare without defining
<code>#define</code> MACRO	preprocessor	textual substitution in <code>.ii</code> – no symbol, no type safety

Linkage and Scope

- TU == translation unit
- Internal Linkage
 - `static` for internal linkage
 - unnamed namespace for internal linkage
- External Linkage
 - `extern` for external linkage
 - named namespace can be external linkage

```
// api.hpp
#pragma once

extern int value;
void process(int);

// a.cpp
#include "api.hpp" // convention sharing

int value = 42;
void process(int x) {}

// b.cpp
#include "api.hpp" // convention sharing

int main() {
    process(value);
}
```

- `std::atomic<T>` is only lock-free for some T/architectures (`is_lock_free()` tells you). `std::mutex` is always a lock abstraction.
- `dmb ish` is a memory barrier: it enforces ordering/visibility of memory accesses across cores/observers before later accesses proceed.
- `atomic<BigStruct>` falls back to mutex for speed

function pointer

```
__BEGIN_DECLS
void(*signal(int, void (*)(int)))(int);
```

```
__END_DECLS
#endif /* !_SYS_SIGNAL_H_ */
```

- `void(*) (int)` is a **function pointer**

Pointer vs reference vs value

```
int x = 5;           // value on STACK – owns data directly
int* p = &x;         // pointer on STACK – holds address of x (8 bytes on 64-bit)
int& r = x;          // reference – alias, not a separate object, no extra memory

int* h = new int(10); // pointer on STACK (8 bytes) → data on HEAP (4 bytes)
```

`sizeof(std::vector<int>)` gives the stack footprint (~24 bytes: `ptr+size+cap`), NOT the heap data size. `.size()` for the logical element count.

Polymorphism

Containers keywords : struct, class, and namespace

- struct is public by default; class is private by default
- struct defaults to public inheritance; class defaults to private inheritance `class Stm32Gpio : public IGpio {`
 - `public : IGpio& ref = stm32_obj;` → private does not allow this
- **namespace** can be extended anywhere
- **namespace** cannot have **template** parameters, unlike the locally closed struct

Valued Template compile time polymorphism

overriding, hiding, and dispatch mode

- **virtual** prefixed base method can be overridden by derived method
- **virtual** `uint64_t micros() = 0;` → `=0` means pure virtual where base cannot be instantiated
- **override**-suffixed derived method lets compiler safety check against wrong paras
- Overriding
 - **virtual** base fns + **override** derived fns → **virtual dispatch** with `vptr` and `vtable`
- Hiding
 - **non-virtual** base fns + same-name derived fns → calls bind by **static type** of the expression
 - **static type** determination
 - `Stm32Gpio obj; obj.set(true);` → derived
 - `IGpio& r = obj; r.set(true);` → base
 - `IGpio* p = &obj; p->set(true);` → base

```
struct IGpio {
    void set(bool);
```

```

    void set(int);
};

struct Stm32Gpio : IGpio {
    using IGpio::set;    // re-expose base overload set
    void set(double);    // additional overload
};

```

vtable layout and virtual dispatch cost

```

Stm32Gpio obj(reg, 5);    // stack object
IGpio* p = &obj;          // base pointer view
IGpio& rb = obj;          // base reference view
Stm32Gpio& rd = obj;      // derived reference view

p->set(true);    // virtual dispatch only if IGpio::set is virtual
rb.set(true);    // same rule as above
rd.set(true);    // statically known derived call (often direct/devirtualized)

// .rodata (compiled into binary, lives forever):
// [
// | vtable for Stm32Gpio:
// |   [0] → Stm32Gpio::~~Stm32Gpio() (.text)
// |   [1] → Stm32Gpio::set()         (.text)
// |   [2] → Stm32Gpio::read()        (.text)
// |
// ]

// Heap / Stack
// [
// | vptr → vtable above | 8 bytes (hidden first field)
// | reg_ = 0x40020014    | 8 bytes
// | pin_ = 5             | 1 byte + 7 padding
// ]

```

dependency injection → polymorphism at consumer

```

constexpr uint32_t GPIOA_ODR = 0x40020014;
Stm32Gpio led(GPIOA_ODR, 5);

// Compile time static dispatch
void blink(Stm32Gpio& led) {led.set(true); }
// Runtime virtual dispatch if base fn is virtual
void blink(IGpio& led) {led.set(true); }

```

duck-typing compile time polymorphism

- Input type should implement `led` or error out

```
template<typename Gpio>
void blink(Gpio& led) {led.set(true); }
```

```
// valued template !
template<uint32_t Addr>
struct Reg {
    // & makes ref() a lvalue alias to the memory at Addr
    static volatile uint32_t& ref() {
        return *reinterpret_cast<volatile uint32_t*>(Addr);
    }
    static void set_bits(uint32_t mask) { ref() |= mask; }
    static void clear_bits(uint32_t mask) { ref() &= ~mask; }
};

// constexpr guarantees compile-time fixed, thus can be templated
constexpr uint32_t GPIOA_ODR = 0x40020014;
Reg<GPIOA_ODR>::set_bits(1 << 5); // set pin 5 high
```

CRTP compile time polymorphism

- GpioBase and GpioBase are different essentially
- `*this` dereferences the base pointer to get a reference. `static_cast<Derived&>(*this)` casts the base *reference* to a derived reference. Without `*`, you'd cast a pointer, which is also valid (`static_cast<Derived*>(this)`) but less idiomatic for member access.

```
template<typename Derived>
class GpioBase {
public:
    // toggle body is identical for all derived
    void toggle() {
        // cast base [this] -> derived [self]
        auto& self = static_cast<Derived&>(*this);
        self.set(!self.read());
    }
};

class Stm32Gpio : public GpioBase<Stm32Gpio> {
public:
    void set(bool high) { /* register write */ }
    bool read() { return /* register read */; }
    // toggle() inherited from GpioBase - calls Stm32Gpio::set/read directly
};
```

Stack, Heap, and Object Lifecycle

Fundamentals

- **Stack** (~1 instruction each, ~100× faster)
 - Allocate: `sub sp, #size`
 - Free: `add sp, #size`
 - `reference's hidden pointer` and a `heap pointer` live on the stack.
- **Heap** (function call → allocator bookkeeping)
 - Allocate: `bl _malloc` → free-list search, possible `mmap` syscall
 - Free: `bl _free` → coalesce free-list, return block
 - `bl` is ARM "Branch with Link": jump to function and store return address in `LR`.
 - Heap overhead is primarily the free-list search + metadata bookkeeping in `malloc/free`, not virtual memory paging. The allocator must traverse the free-list, split/coalesce blocks, and maintain headers. Stack is just a pointer bump (`sub sp`).

Stack Frame

- Every function call pushes a **frame** onto the stack containing:
 - 1. return address 2. args 3. local variable in stack
- All stack `locals` deleted with frame returned
- In-fn `int* p = new int(5);`, only `p` (the pointer) is destroyed at return.
 - `delete p` or transfer to `smart pointer` explicitly before memory leak

call chain: `main()` → `foo()` → `bar()`

Stack (grows downward):

	high address
main() frame	x, v, s, gpio
foo() frame	local, temp
bar() frame	i, j ← sp (stack pointer) points here
	low address

`bar()` returns → `sp` moves up → `bar's` frame is gone

`foo()` returns → `sp` moves up → `foo's` frame is gone

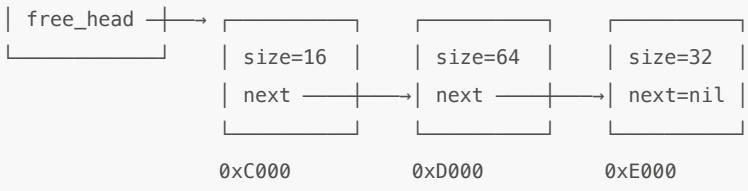
Heap, free-list

The allocator (`libmalloc`) maintains a **free-list** – a `linked list` stored inside freed heap blocks:

- `head` pointer: lives in allocator globals (`.data`)
- `next` pointers: live inside each freed block (reusing the now-unused data area)
- alternative DS for `linked list` : for performance/predictability include segregated size bins, buddy allocators, slab/pool allocators, and TLSF (good bounded latency for RT systems).

`.data` (allocator global) HEAP (freed blocks)

┌───────────┐



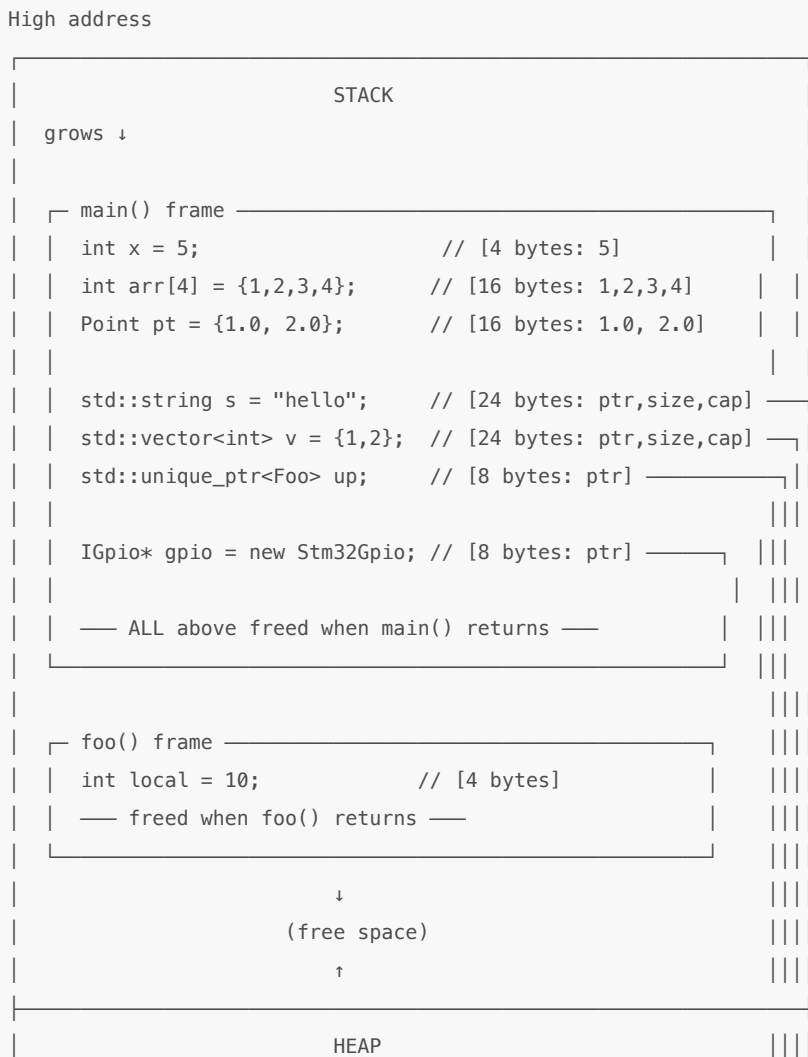
- `malloc(20)`: walk free-list $\rightarrow$ find block $\geq 20B \rightarrow$ split $\rightarrow$ return pointer
- `free(ptr)`: mark block unused $\rightarrow$ write `next` pointer into it $\rightarrow$ re-link list

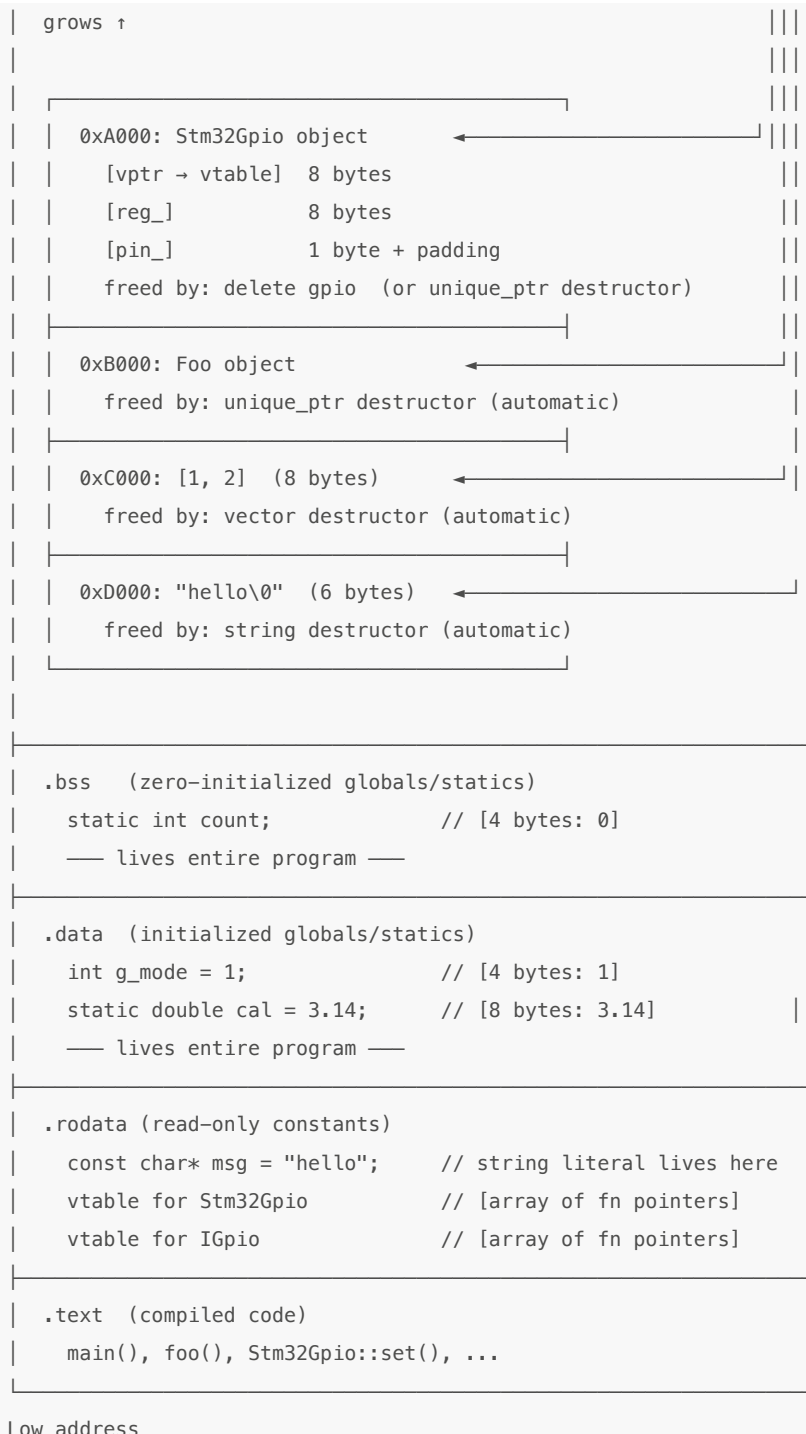
When do we need data on heap

Stack is preferred but:

1. **Lifetime must outlive the scope** – returning a large object from a factory, storing in a container that persists after the creating function returns.
2. **Size unknown at compile time** – `new int[n]` where `n` is runtime. Stack requires compile-time size (VLAs are non-standard).
3. **Too large for stack** – stack is typically 1-8 MB. A 10 MB array must go on heap.
4. **Virtual Dispatch**

Full memory layout – where everything lives





Rules of thumb

- Local variable → **stack** (automatic, fast, no leak possible)
- **global** / **static** → **.data** or **.bss** (lives forever)
 - **global** has external linkage (visible in all TUs via **extern**);
 - **static** at file scope has internal linkage (visible only in this TU).
 - TU (**.ii**) = Translation Unit
 - Both live in **.data/.bss** for the program's lifetime.
- **static** function code is in **.text**; its local variables are still stack unless declared **static**.
- **new** / **malloc** → heap (manual or smart-pointer managed)
- String literals ("hello") → **.rodata** (compiled in, read-only)
- vtables → **.rodata** (one per class, compiled in)

- Code → .text (compiled in, read-only, executable)

Copy vs move

Trivial types – memcpy is the entire operation

```
int a = 5;
int b = a;           // compiler emits: memcpy(&b, &a, 4) – done
```



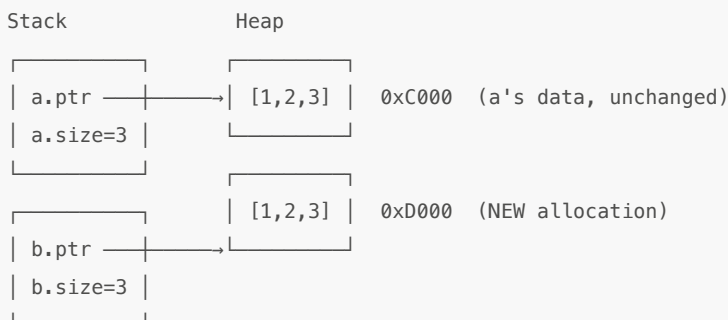
Non-trivial types – constructor must run, NOT memcpy

```
std::vector<int> a = {1, 2, 3};

// Stack           Heap
// [ a.ptr ]       [ [1,2,3] ] 0xC000
// [ a.size=3 ]
// [ a.cap=3 ]
```

- `std::vector` is NOT a linked list. It is a contiguous dynamic array (single heap block).
- `std::list` is the doubly-linked list.
- Vector gives $O(1)$ random access; list gives $O(1)$ insert/remove at iterators.

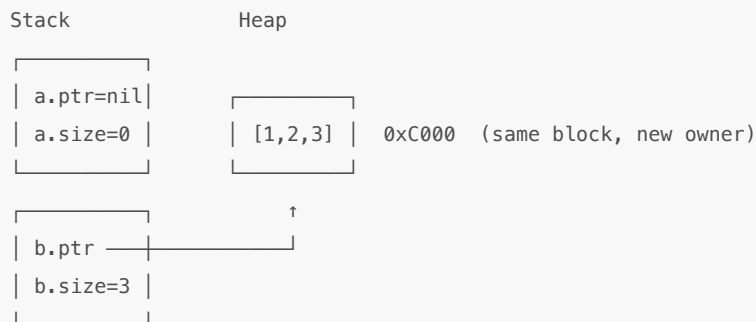
Copy: `vector<int> b = a;`



- Copy constructor: allocate new heap block, memcpy the CONTENTS to the heap
- Cost: malloc() + memcpy(N elements)
- `malloc` returns a *new and distinct* heap address from the free-list.

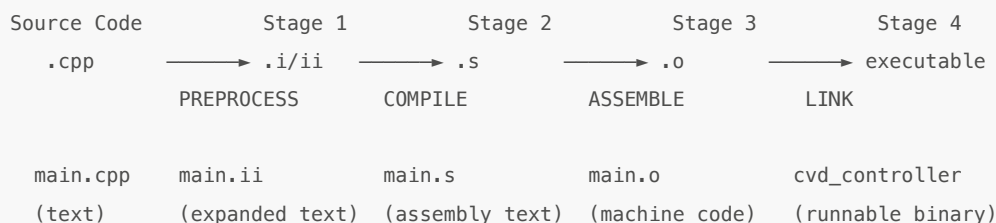
- Now `a.ptr` and `b.ptr` point to different blocks – no double free.

Move: `vector<int> b = std::move(a);`



- Move constructor: steal pointer, null source
- Cost: 3 pointer/int assignments (24 bytes)
- `a` is still alive and valid – just empty

C++ Build Pipeline



Stage 1: PREPROCESS – `cpp` (or `g++ -E`)

- `#include` parts of `.cpp` are expanded by including `.h` into `.ii`
 - `.cpp` inclusions causes duplicate definitions, slow builds, and poor dependency hygiene.
- CMake's `target_include_directories` becomes the `-I` flag
- `cpp -I src/common src/controller/main.cpp -o main.i`
- `g++ -E -I src/common src/controller/main.cpp -o main.i`

Stage 2: COMPILE – `g++ -S`

- `g++ -S -std=c++17 -I src/common src/controller/main.cpp -o main.s`

Stage 3: ASSEMBLE – `g++ -c` (or `'as'`)

- `g++ -c -std=c++17 -I src/common src/controller/main.cpp -o main.o`

Stage 4: LINK – `g++` (calls `ld` internally)

- ODR violations are often detected at link time for duplicate non-inline definitions, but many ODR violations are not diagnosable and become runtime undefined behavior.

- The dynamic loader path is stored in the ELF `PT_INTERP` segment (e.g., `/lib64/ld-linux-x86-64.so.2` on Linux, `/usr/lib/dyld` on macOS). The kernel reads this at exec time to determine which loader runs.
- `g++ main.o -o main`
 - `-static` → link libraries statically but still can fail due to `OS`
 - `container` solves userspace issue not kernel issue
- `file main` → check the dynamic loader

All at once:

- `g++ -std=c++17 -I src/common src/controller/main.cpp -o main`

Compilation details

- Lexing & Parser
 - Tokenize source → build Abstract Syntax Tree (AST)
 - tree structure reveals the sequence for execution
- Semantic analysis
 - type checking, access control
 - Verify `pkt.temperature` is double, `send()` args match signature
- Template instantiation
 - replace the `template <typename T>` arg
- Overload resolution
 - `std::cout << pkt.temperature` → pick operator<<(double)
- Optimization (`-O2`)
 - Inline functions, eliminate dead code, unroll loops, vectorize
- Register allocation
 - Map variables to CPU registers (`rax`, `xmm0`, etc.)
- Instruction selection
 - Choose x86 instructions (`movsd`, `addsd`, `subsd`)
- Struct layout
 - Compute field offsets: `temperature` is at byte 12 from struct start
- Stack frame layout
 - Decide where each local variable lives relative to Register Stack Pointer

inline, constexpr, and inlining

- `inline`, `constexpr`, `template` all produce weak symbols → all ODR-safe in headers
 - each `.o` carries a copy → linker keeps one → discards duplicates
 - non-inline non-template function/variable definitions in headers usually cause multiple-definition/ODR problems
- `constexpr` → implies `inline` (weak symbol) + CAN evaluate at compile time
 - `constexpr int x = factorial(5);` → computed by compiler, no runtime call
 - runtime args → falls back to normal (inlined) function call
- Inlining (body expanded at call site, no `bl`) is a compiler optimization
 - `-O2` inlines small fns automatically regardless of `inline` keyword
 - too much inlining bloats code → cache misses → slower
 - `-O2` also folds obvious compile-time constants
 - `volatile` prevents reordering or cutting of loads/stores of a var

Link (ld)

- Linker resolves compile-yielded symbols by name – it does not re-check types.

1. Link time

- Linker marks executable as dynamically linked.
- Statically linked build resolves library code into the executable at link time (no runtime shared-library loading for those libs, larger binary, fewer runtime deps).
- writes metadata to executable
 - PT_INTERP (loader path, e.g. ld-linux-x86-64.so.2)
 - DT_NEEDED entries (required shared libs)

2. Runtime (after build, when you run executable)

- Kernel reads PT_INTERP and starts the dynamic loader.
- Loader resolves shared libraries and verifies arch / version

.o / .so / .dylib / .dll comparison

- **.o** – relocatable object file (one TU's machine code, unresolved symbols, build-time only)
- **.so** – Linux shared library (linked, PIC, runtime-loadable via **dlopen**)
- **.dylib** – macOS shared library (equivalent to **.so**)
- **.dll** – Windows shared library (equivalent to **.so**)
- **ctypes.CDLL** – Python's FFI wrapper that calls **dlopen/LoadLibrary** to load a **.so/.dylib/.dll**
- **ldd** – Linux CLI tool that prints shared library dependencies of an executable (macOS equivalent: **otool -L**)

Property	.o (object file)	.so / .dylib / .dll (shared library)
Symbols	unresolved (U = undefined)	all resolved
Runtime loadable	no	yes (dlopen / ctypes.CDLL / LoadLibrary)
Contents	one .cpp's machine code	linked, complete library
Position-independent	not required	yes, typically needs -fPIC
Primary use	linker (ld) at build time	OS loader at runtime

.so properties:

- **extern "C"** for Python/C interop (no mangling)
- **g++ -std=c++17 -shared -fPIC pid_lib.cpp -o libpid.dylib**
- **-fPIC** = generate Position-Independent Code (code that can be loaded at arbitrary addresses, required for shared libraries on most platforms).

Mangling and symbols

- Mangling distinguishes overloaded function names in C++

```
_ZN3PID7computeEddd
|| | |   |||
|| | |   |—— d = double (paras)
```

```

|| | | _____ E = end of nested name
|| | | _____ 7compute = method name
|| | _____ 3PID = class name
| _____ N = nested name start
_____ _Z = C++ mangled

```

- `extern "C" void handle_signal(int sig);` disables mangling so the symbol is `handle_signal` only
- `nm main.o`
 - `T` global function / `t` static (internal linkage) function
 - `b` static local data
 - `r` string literals and constants
 - `U` undefined – should be linked at link time
 - `W` weak symbols – inline, constexpr, template
- weak symbols
 - inline functions are intrinsically weak
 - linker keeps one COMDAT-equivalent copy for inline/template definitions that are ODR-equivalent

Header files & ODR

- `<T>` must be in `.h` because compiler needs the body to instantiate per type
 - escape hatch: explicit instantiation in one `.cc` keeps template body out of header
 - `.cc` is just a C++ source-file extension (same role as `.cpp/.cxx` by convention).
- any non-template definition in `.h` must be marked `inline` to be ODR-safe
 - class member functions defined inside class body are implicitly `inline`
- `#ifndef F00_H_ / #define F00_H_ / #endif` guard prevents multiple inclusion
 - The first `#include` seen by the preprocessor wins.
- one `.h` for all `.cpp` can avoid ODR (One Definition Rule)
 - Bazel / CMake guarantees single header version so ODR is avoided
- two `.cpp` files include different versions of the same header (e.g., struct layout changed), each `.o` is compiled with a different definition of the same symbol. The linker silently picks one – the other `.o` now interprets memory with the wrong layout → undefined behavior (silent corruption, not a linker error).

`__restrict`

- these pointers do not reference one another during function call
- do not bl hence compiler friendly

```

int getaddrinfo(const char * __restrict, const char * __restrict,
               const struct addrinfo * __restrict,
               struct addrinfo ** __restrict);

```

g++ invocation

```
g++ -v -std=c++17 -I src/common src/controller/main.cpp -o main 2>&1
Apple clang version 17.0.0 (clang-1700.6.3.2)
Target: arm64-apple-darwin25.4.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
```

-std=c++17 & POSIX & libc

- std flag specifies which path stds should be found
- POSIX libs are OS specific and can be checked by `g++ -###`

	std:: (C++)	libc (C)	POSIX
Defined by	ISO C++ committee	ISO C committee	IEEE / Open Group
Name mangling	yes (<code>_ZSt5clamp...</code>)	no (<code>memcpy</code>)	no (<code>signal</code>)
Namespace	<code>std::</code>	<code>std::</code> or global	global only
Templates/overloads	yes	no	no (supports both C/C++)
Talks to kernel	rarely	sometimes (<code>malloc</code>)	almost always
Portable to Windows	yes (MSVC)	yes (MSVC)	no

Embedded C++ Patterns

1. RAII scope guard – auto-release hardware resources

```
struct CriticalSection {
    CriticalSection() { __disable_irq(); }
    ~CriticalSection() { __enable_irq(); }
};

void transfer() {
    CriticalSection cs;           // interrupts disabled
    shared_buf[idx++] = data;
}                                // ~CriticalSection() re-enables – even if early return

// Generic scope guard (C++17)
template<typename F>
struct ScopeGuard {
    F fn;
    ~ScopeGuard() { fn(); }
};

// usage: ScopeGuard guard{[] { HAL_SPI_Release(); }};
```

2. ISR ↔ main communication

```

// ISR sets flag – main polls
volatile bool data_ready = false;

// ISR (runs in interrupt context – no heap, no blocking)
extern "C" void USART1_IRQHandler() {
    rx_buf.push(USART1->DR); // lock-free ring buffer
    data_ready = true;
}

// main loop
while (true) {
    if (data_ready) {
        data_ready = false;
        process(rx_buf.pop());
    }
    __WFI(); // sleep until next interrupt
}

```

3. Ring buffer – lock-free, fixed-size, ISR-safe

```

template<typename T, size_t N>
struct RingBuffer {
    static_assert((N & (N - 1)) == 0, "N must be power of 2");
    T buf[N]{};
    volatile size_t head = 0; // written by producer (ISR)
    volatile size_t tail = 0; // written by consumer (main)

    bool push(T val) {
        size_t next = (head + 1) & (N - 1);
        if (next == tail) return false; // full
        buf[head] = val;
        head = next;
        return true;
    }

    bool pop(T& out) {
        if (head == tail) return false; // empty
        out = buf[tail];
        tail = (tail + 1) & (N - 1);
        return true;
    }
};

```

4. Static / pool allocation – no heap

```

// Fixed pool – no malloc, no fragmentation
template<typename T, size_t N>
struct Pool {
    std::aligned_storage_t<sizeof(T), alignof(T)> storage[N];

```

```

bool used[N]{};

T* alloc() {
    for (size_t i = 0; i < N; ++i)
        if (!used[i]) { used[i] = true; return reinterpret_cast<T*>(&storage[i]); }
    return nullptr;
}

void free(T* p) {
    size_t i = reinterpret_cast<std::aligned_storage_t<sizeof(T), alignof(T)>*>(p) - storage;
    p->~T();
    used[i] = false;
}

};

Pool<Packet, 16> pkt_pool; // 16 Packets, all on stack/static

```

5. Placement new – construct at specific memory

```

// DMA buffer at fixed address
alignas(32) uint8_t dma_buf[sizeof(Packet)];

Packet* p = new (dma_buf) Packet{.id = 1, .temp = 36.5}; // no malloc
// ... use p ...
p->~Packet(); // explicit destructor, no delete

```

6. State machine – enum + transition function

```
enum class State { Idle, Heating, Stabilize, Processing, Fault };
enum class Event { Start, TempReached, Done, Error, Reset };

State transition(State s, Event e) {
    switch (s) {
        case State::Idle:
            if (e == Event::Start) return State::Heating;
            break;
        case State::Heating:
            if (e == Event::TempReached) return State::Stabilize;
            if (e == Event::Error) return State::Fault;
            break;
        case State::Stabilize:
            if (e == Event::Done) return State::Processing;
            break;
        case State::Fault:
            if (e == Event::Reset) return State::Idle;
            break;
        default: break;
    }
    return s; // no transition
}
```

7. Type-safe units – prevent mixing

```
struct Milliseconds { uint32_t v; };
struct Microseconds { uint32_t v; };
struct Celsius      { double v; };

void delay(Milliseconds ms);           // can't accidentally pass µs
void set_temp(Celsius target);         // can't pass raw double

// compile error: delay(Microseconds{100});
```

8. Double buffering – DMA ping-pong

- DMA with double buffering completes the data read for CPU → saves polling

```
alignas(32) uint16_t adc_buf[2][256]; // two buffers
volatile int active = 0;               // DMA writes to active

extern "C" void DMA1_IRQHandler() {
    active ^= 1;                       // swap
    DMA1->M0AR = (uint32_t)adc_buf[active]; // point DMA to new buffer
}

// main processes the inactive buffer – no race
void process() {
```

```
uint16_t* safe = adc_buf[active ^ 1];
for (int i = 0; i < 256; ++i) { /* use safe[i] */ }
}
```

9. Error handling without exceptions

```
enum class Err { Ok, Timeout, CrcFail, BusError };

struct Result {
    double value;
    Err err;
    explicit operator bool() const { return err == Err::Ok; }
};

Result read_sensor(uint8_t addr) {
    if (!i2c_start(addr)) return {0, Err::BusError};
    uint16_t raw = i2c_read16();
    if (!verify_crc(raw)) return {0, Err::CrcFail};
    return {raw * 0.01, Err::Ok};
}

// usage
if (auto r = read_sensor(0x48); r) {
    temperature = r.value;
} else {
    handle_error(r.err);
}
```

10. Compile-time platform selection

```
template<typename MCU> struct Gpio;

struct Stm32F4 {};
struct Stm32H7 {};

template<> struct Gpio<Stm32F4> {
    static void set(int pin) { GPIOA->BSRR = (1 << pin); }
};

template<> struct Gpio<Stm32H7> {
    static void set(int pin) { GPIOA->BSRR = (1 << pin); }
};

using Led = Gpio<Stm32F4>; // single point of platform change
Led::set(5);
```



